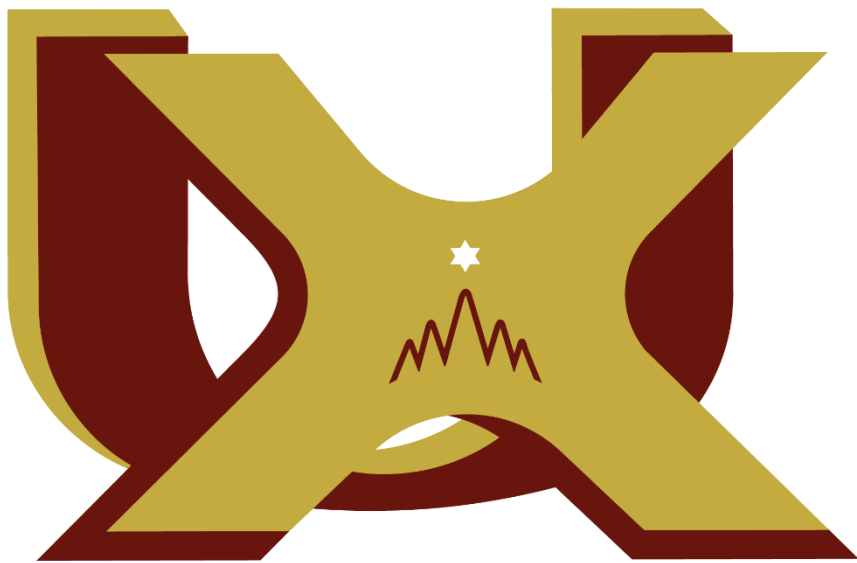


MÉTODOS DE ORDENAMIENTO

Héctor Luis García Marthen

Universidad Xalapa



UNIVERSIDAD DE XALAPA

• ————— •
Saber / Trascender

¿Qué es un algoritmo de ordenamiento?

Es un conjunto finito de instrucciones paso a paso diseñado para reorganizar los elementos de una colección (como un arreglo o lista) en un orden predeterminado según un criterio específico.

¿Cuál es el objetivo principal del ordenamiento?

Optimizar la organización de la información para facilitar y acelerar su recuperación, consulta, análisis y visualización posterior.

¿Qué tipos de datos pueden ordenarse?

Cualquier tipo de dato que posea un operador de comparación válido o un orden total definido:

- **Numéricos:** Enteros (int), flotantes (float).
- **Texto:** Cadenas de caracteres (string), ordenadas lexicográficamente o alfabéticamente.
- **Fechas y tiempo:** Estampas de tiempo (timestamp), fechas (date).
- **Estructuras/Objetos complejos:** Clases o registros ordenados según un atributo específico (clave de ordenamiento, como ID, apellido o precio).

¿Qué significa ordenar de forma ascendente?

Disponer los elementos de menor a mayor valor ($A \leq B \leq C$ o $1 \leq 2 \leq 3$). En texto corresponde al orden alfabético ($A \rightarrow Z$).

¿Qué significa ordenar de forma descendente?

Disponer los elementos de mayor a menor valor ($C \geq B \geq A$ o $3 \geq 2 \geq 1$). En texto corresponde al orden alfabético inverso ($Z \rightarrow A$).

¿Por qué es importante ordenar los datos antes de realizar determinadas búsquedas?

Porque algoritmos de búsqueda altamente eficientes, como la **búsqueda binaria**, requieren obligatoriamente que la estructura esté ordenada. La búsqueda secuencial en un conjunto desordenado requiere una complejidad de $O(n)$, mientras que sobre datos ordenados la búsqueda binaria la reduce a $O(\log n)$, optimizando drásticamente el uso de CPU al consultar grandes volúmenes de información.

¿Qué características deberían considerarse al elegir un algoritmo de ordenamiento?

1. **Complejidad temporal:** Desempeño en el peor, promedio y mejor de los casos.
2. **Complejidad espacial:** Cantidad de memoria auxiliar requerida (algoritmos *in-place* vs. los que requieren memoria externa).
3. **Estabilidad:** Si mantiene el orden relativo original de los elementos que tienen claves de ordenamiento idénticas.
4. **Tamaño y estado del conjunto:** Cantidad de datos y si están casi ordenados o totalmente desordenados.

Sorting Algorithms	Time Complexity			Space Complexity
	Best Case	Average Case	Worst Case	Worst Case
Bubble Sort	$O(N)$	$O(N^2)$	$O(N^2)$	$O(1)$
Selection Sort	$O(N^2)$	$O(N^2)$	$O(N^2)$	$O(1)$
Insertion Sort	$O(N)$	$O(N^2)$	$O(N^2)$	$O(1)$
Quick Sort	$O(N \log N)$	$O(N \log N)$	$O(N^2)$	$O(N)$
Merge Sort	$O(N \log N)$	$O(N \log N)$	$O(N \log N)$	$O(N)$
Heap Sort	$O(N \log N)$	$O(N \log N)$	$O(N \log N)$	$O(1)$

Complejidades temporales y espaciales de algoritmos. Fuente:
AfterAcademy

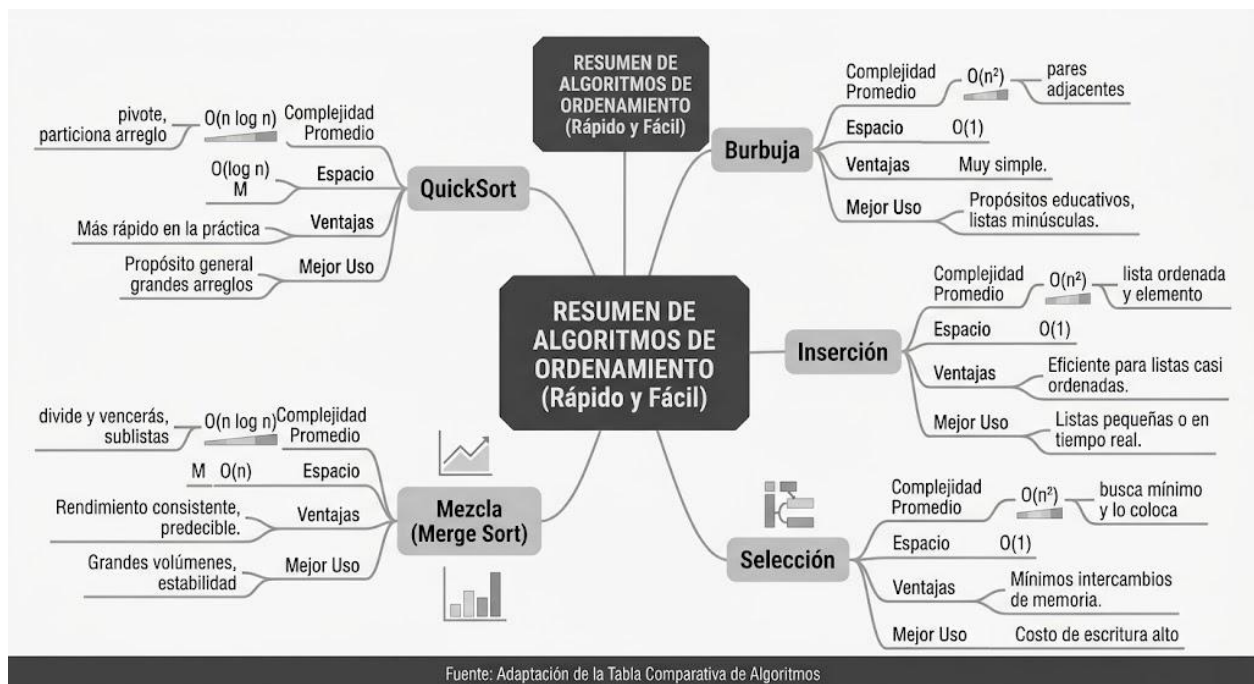
TABLA COMPARATIVA

TABLA DE COMPARACIÓN SIMPLE DE ALGORITMOS DE ORDENAMIENTO (Rápido y Fácil)

ALGORITMO		COMPLEJIDAD PROMEDIO	ESPACIO	VENTAJAS PRINCIPALES	MEJOR USO
Burbuja	pares adyacentes, valores flotan	$O(n^2)$	$O(1)$	• Muy simple.	• Propósitos educativos, listas minúsculas.
Inserción	lista ordenada, inserta un elemento	$O(n^2)$	$O(1)$	• Eficiente para listas casi ordenadas.	• Listas pequeñas o en tiempo real.
Selección	Busca el mínimo, lo coloca al final	$O(n^2)$	$O(1)$	• Mínimos intercambios de memoria.	• Cuando el costo de escritura en memoria es alto.
Mezcla (Merge Sort)	Divide y vencerás, combina sublistas ordenadas	$O(n \log n)$	$O(n)$ M	• Rendimiento consistente y predecible.	• Grandes volúmenes, estabilidad requerida.
QuickSort	Elige un pivote, particiona el arreglo	$O(n \log n)$	$O(\log n)$ M	• El más rápido en la práctica.	• Propósito general para arreglos grandes.

Fuente: Resumen de Comparación de Algoritmos

Mapa mental



Fuente: Adaptación de la Tabla Comparativa de Algoritmos

Detalles por Algoritmo

1. Ordenamiento de Burbuja (Bubble Sort)

- **Funcionamiento:** Recorre la lista repetidamente comparando elementos contiguos e intercambiándolos si no están en orden.
- **Estrategia:** Comparación secuencial iterativa y permutación directa.
- **Complejidad:** Temporal $O(n^2)$ (Promedio/Peor), $O(n)$ (Mejor); Espacial $O(1)$.
- **Ventajas:** Extremadamente simple, detecta si la lista está ordenada rápidamente.

- **Desventajas:** Muy lento; número elevado de operaciones de escritura.

- **Características:** Estable, *in-place*, de paso iterativo.

2. Ordenamiento por Selección (Selection Sort)

- **Funcionamiento:** Encuentra el elemento mínimo no ordenado y lo intercambia con el primer elemento desordenado de la lista.
- **Estrategia:** Selección greedy/avara del mínimo elemento por pasada.
- **Complejidad:** Temporal $O(n^2)$ (Todos los casos); Espacial $O(1)$.
- **Ventajas:** Mínimo número de intercambios de memoria ($O(n)$).
- **Desventajas:** Rendimiento constante ineficiente $O(n^2)$ sin importar el orden inicial.
 - **Características:** Inestable (por defecto), *in-place*, no adaptable.

3. Ordenamiento por Inserción (Insertion Sort)

- **Funcionamiento:** Extrae un elemento de la sección no ordenada y lo desplaza hacia la izquierda hasta situarlo en la posición correcta.
- **Estrategia:** Crecimiento progresivo de un subconjunto ordenado.
- **Complejidad:** Temporal $O(n^2)$ (Peor/Promedio), $O(n)$ (Mejor); Espacial $O(1)$.
- **Ventajas:** Altamente eficiente para conjuntos pequeños o casi ordenados; en línea.
- **Desventajas:** Ineficiente para arreglos grandes desordenados.

- **Características:** Estable, *in-place*, adaptable.

4. Ordenamiento por Mezcla (Merge Sort)

- **Funcionamiento:** Divide la lista en dos mitades recursivamente, las ordena individualmente y luego intercala las dos sublistas ordenadas.
- **Estrategia:** Divide y Vencerás (*Divide and Conquer*).
- **Complejidad:** Temporal $O(n \log n)$ (Todos los casos); Espacial $O(n)$.
- **Ventajas:** Desempeño uniforme $O(n \log n)$ garantizado; ideal para datos masivos o externos.
 - **Desventajas:** Consume memoria auxiliar $O(n)$.
- **Características:** Estable, no *in-place* (requiere memoria externa), recursivo.

5. QuickSort (Ordenamiento Rápido)

- **Funcionamiento:** Elige un pivote, reorganiza la lista de forma que los valores menores queden a un lado y los mayores al otro, y aplica el proceso recursivamente a las particiones.
- **Estrategia:** Divide y Vencerás basada en particionado con pivote.
- **Complejidad:** Temporal $O(n \log n)$ (Promedio/Mejor), $O(n^2)$ (Peor caso); Espacial $O(\log n)$.
- **Ventajas:** El algoritmo en memoria más rápido en la práctica; bajo consumo de espacio extra.
- **Desventajas:** Sensible a la elección del pivote; inestable.
 - **Características:** Inestable, *in-place*, recursivo.

Referencias (Formato APA 7.^a ed.)

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4.^a ed.). MIT Press.
- Knuth, D. E. (1998). *The art of computer programming: Vol. 3. Sorting and searching* (2.^a ed.). Addison-Wesley Professional.
 - Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4.^a ed.). Addison-Wesley Professional.
- Weiss, M. A. (2014). *Data structures and algorithm analysis in Java* (3.^a ed.). Pearson.